

Ponytail — Setup Guide

A free coding-agent skill that builds only what the task actually needs — while keeping validation, error handling, security, and accessibility fully intact.

GitHub

DietrichGebert/ponytail

License

MIT

Verified Results (real Claude Code benchmark, 12 tasks)

-54% code written | -20% cost | -27% faster | 100% safety checks kept

Supported

20 AI coding agents, including Claude Code, Codex, Copilot, Gemini CLI, Cursor, and more

1. Installation — by Agent

Claude Code

Run as two separate prompts (also works in Claude Code Desktop's Code tab):

```
/plugin marketplace add DietrichGebert/ponytail  
/plugin install ponytail@ponytail
```

Codex

```
codex plugin marketplace add DietrichGebert/ponytail  
codex plugin add ponytail@ponytail
```

Then run codex, open /hooks, review and trust the two lifecycle hooks, then start a new thread.

GitHub Copilot CLI

```
copilot plugin marketplace add DietrichGebert/ponytail  
copilot plugin install ponytail@ponytail
```

Or inside Copilot CLI: /plugin marketplace add DietrichGebert/ponytail then /plugin install ponytail@ponytail.

Gemini CLI

```
gemini extensions install https://github.com/DietrichGebert/ponytail
```

Ponytail becomes always-on context for your sessions.

Antigravity CLI

```
agy plugin install https://github.com/DietrichGebert/ponytail
```

Uses the same Gemini extension under the hood.

Cursor

```
git clone https://github.com/DietrichGebert/ponytail  
node ponytail/scripts/cursor-hooks.js install
```

Open a new Cursor chat, then switch modes anytime with /ponytail lite, full, ultra, or off. Easier alternative: copy the matching Ponytail rule into .cursor/rules/ instead of running the script.

Other supported agents

Di

```
pi install git:github.com/DietrichGebert/ponytail
```

OpenCode

Add to opencode.json: { "plugin": ["@dietrichgebert/ponytail"] }

Qoder

Works via AGENTS.md automatically, or copy .qoder/rules/ponytail.md

Hermes Agent

```
hermes plugins install DietrichGebert/ponytail --enable (then restart Hermes)
```

Devin CLI

```
devin plugins install DietrichGebert/ponytail
```

OpenClaw

```
clawhub install ponytail
```

Grok Build

```
grok plugin install DietrichGebert/ponytail --trust
```

Rule-based setups also available for: Windsurf, Cline, GitHub Copilot Chat, Aider, Kiro, Zed, CodeWhale, Swival, and Qoder — copy the matching rule file from the Ponytail repo into each tool's config location.

2. Commands & Decision Ladder

Useful commands (on hosts with command support)

<code>/ponytail</code>	Check the current mode
<code>/ponytail lite full ultra off</code>	Change the intensity level
<code>/ponytail-review</code>	Review your current diff for over-engineering
<code>/ponytail-audit</code>	Audit the entire repository, not just the diff
<code>/ponytail-gain</code>	Show the measured impact (code/tokens/cost saved)

The decision ladder

Once installed, Ponytail runs every coding request through this ladder before writing anything:

1. Does this need to exist?
2. Is it already in the codebase?
3. Does the standard library do it?
4. Does the platform already provide it?
5. Is there already a dependency for it?
6. Can one line solve it?
7. Only then → build the minimum required.

Important: minimal doesn't mean unsafe. Ponytail explicitly keeps validation, error handling, security, and accessibility intact at every step of the ladder — it only cuts unnecessary code, not necessary safeguards.

**“Don't write less code.
Write only the code that needs to exist.”**

That's the core idea behind Ponytail.

Notes: Numbers above come from Ponytail's own published benchmark (real Claude Code sessions on a FastAPI + React repo, 12 feature tasks, same agent with/without the skill, Haiku 4.5). Results can vary by codebase and task type — savings are largest where genuine over-engineering exists, and shrink toward zero on already-minimal code. GitHub: DietrichGebert/ponytail. License: MIT.